502094 - Software Deployment, Operations And Maintenance

Module 12: Automating Server Configuration with Ansible & Terraform

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Module Overview

- **Goal**: understand **Configuration Management (CM)** and how to automate server configuration reliably
- Focus tools and concepts:
  - CM vs IaC vs simple shell scripts
  - **Ansible** as primary CM tool
  - Integration with **Terraform** and CI/CD
- Teaching style:
  - Theory → examples → concrete playbooks
- Outcome:
  - Students can automate setup of Node.js / Spring Boot servers using Ansible

1. Introduction to Configuration Management

- **Configuration Management (CM)**:
  - Practice of defining and maintaining consistent **system configuration** across servers
- Examples of configuration:
  - Installed packages, System users and permissions
  - Application runtimes (Java, Node.js)
  - Services and configuration files (Nginx, systemd)
- CM tools replace:
  - Manual SSH commands
  - “One-off” scripts lost in someone’s laptop

Why Configuration Management Matters

- Without CM:
  - Servers hand-configured manually
  - No clear record of changes
  - Hard to reproduce environments
- With CM:
  - **Repeatable** server setup
  - **Consistent** across dev/staging/production
  - Easier audit and investigation (what changed, when)
- For DevOps: CM is a foundation for **reliable deployments** and **incident response**

Imperative vs Declarative Configuration

- **Imperative** (how to do it):
  - Shell script:

    ```
    apt update
    apt install -y nginx
    systemctl enable nginx
    systemctl start nginx
    ```

  - We specify **steps** to reach state, risk of drift if steps change
  - We have to handle idempotency manually

Imperative vs Declarative Configuration

- **Declarative** (what final state should be):
  - Ansible task:

```
- name: Ensure nginx is installed and running
  apt:
    name: nginx
    state: present
  notify: restart nginx
```

  - We specify **desired state**, Ansible decides steps, ensures idempotency
  - Easier to maintain and reason about
  - Encourages best practices

Manual Setup vs Automated Configuration

- **Manual setup**:
  - SSH into server
  - Run commands from memory or a doc
  - Risk of typos, missing steps
- **Automated configuration**:
  - Declare desired state in a playbook
  - Run `ansible-playbook` to enforce it
  - Idempotent: safe to run many times
- Benefit:
  - **Zero-drift environments**: Servers converge to the same configuration

CM vs IaC vs Script Automation

- **IaC (e.g., Terraform)**:
  - Manages **infrastructure resources**: VMs, networks, load balancers
- **Configuration Management (e.g., Ansible)**:
  - Manages **inside the server**: Packages, files, services, applications
- **Script automation (shell/Python)**:
  - Flexible, but often:
    - Non-idempotent
    - Harder to maintain and reuse
- Real-world workflow:

  i. Terraform **creates** servers

  ii. Ansible **configures** servers

2. Why Ansible?

- Reasons Ansible is popular:
  - **Agentless**:
    - Uses SSH, no agent to install on nodes
  - **YAML-based**:
    - Human-readable, easy to review in code
  - **Cross-platform**:
    - Linux, some macOS, network appliances, cloud services
  - **Cloud-agnostic**:
    - Works with AWS, Azure, GCP, on-prem
- In this course:
  - Terraform: infrastructure `and` Ansible: configuration & app deployment

Ansible in a DevOps Workflow

- Typical workflow:
  i. Terraform provisions EC2 instances and security groups
  ii. Terraform outputs IP addresses
  iii. Ansible inventory is generated from Terraform output
  iv. Ansible:
    - Installs packages (Nginx, Node.js, Java)
    - Deploys application code
    - Configures services and firewalls
- Benefits:
  - Clear separation of concerns
  - Fully automated environment bootstrap

3. Installing Ansible & Key Terminology

- We cover:
  - Installing Ansible on macOS/Linux/Windows
  - Required plugins for AWS integrations
  - Checking `ansible --version`
- Then terminology:
  - **Inventory**, **Playbook**, **Tasks**, **Modules**, **Handlers**
  - **Variables**, **Templates**, **Facts**, **Roles**
  - **Galaxy**, **Idempotency**

Installing Ansible (macOS & Linux)

- On **Ubuntu / Debian**:

```
sudo apt update
sudo apt install -y ansible
```

- On **macOS** (Homebrew):

```
brew install ansible
```

- Validate installation:

```
ansible --version
```

- Recommendation: Use a **Python virtualenv** for advanced setups, but system package is fine for this course

Installing Ansible on Windows

- Options:
  - Use **WSL (Windows Subsystem for Linux)** and install Ansible there
  - Or use a Linux VM as Ansible control node
- Approach for course:
  - Students on Windows:
    - Install Ubuntu via WSL
    - Follow Linux installation steps inside WSL
- Ansible typically runs from **Linux control node** even in mixed environments

Required Plugins for AWS

- Ansible integrates with AWS via:
  - `boto3` (Python AWS SDK)
- Install:

```
pip install boto3 botocore
```

- For dynamic inventory:
  - Additional AWS inventory plugins available in Ansible
- AWS credentials:
  - Same as Terraform:
    - `~/.aws/credentials` or environment variables

Key Terminology: Inventory & Playbook

- **Inventory**:
  - List of target hosts (servers) Ansible can manage
  - Can be:
    - Static file (`inventory.ini`, `hosts.yaml`)
    - Dynamic (AWS EC2, etc.)
- **Playbook**:
  - YAML file with **plays**
  - A **play** maps a group of hosts to a set of **tasks**

Key Terminology: Inventory & Playbook

- Example:

```yaml
- hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
```

- Here:
  - `hosts`: target group from inventory
  - `become`: use sudo
  - `tasks`: list of actions to perform

Key Terminology: Tasks, Modules, Handlers

- **Task**:
  - Single unit of work executed on target hosts
- **Module**:
  - Reusable, idempotent unit Ansible uses to perform tasks:
    - `apt`, `service`, `copy`, `template`, `user`, `file`, `git`, `pip`, `docker_container`, etc.
- **Handler**:
  - Special task that runs **only when notified**
  - Often used to restart services:

Key Terminology: Tasks, Modules, Handlers

```yaml
handlers:
  - name: restart nginx
    service:
      name: nginx
      state: restarted
```

- Explain:
  - Handlers defined at play level
  - Notified by tasks when a change occurs

Key Terminology: Variables, Templates, Facts

- **Variables**:
  - Values that can change per host, group, or environment
- **Templates**:
  - Files processed with **Jinja2** to inject variables into configs
  - Example: Nginx config template
- **Facts**:
  - System information gathered by Ansible:
    - OS type, hostname, IP addresses, CPU, memory
  - Accessible via `ansible_facts` in tasks and templates

Key Terminology: Roles, Galaxy, Idempotency

- **Roles**:
  - Structured way to organize playbooks into reusable components
  - Example roles:
    - `common`, `nginx`, `nodejs`, `docker`, `springboot`
- **Galaxy**:
  - Ansible Galaxy: public hub of community roles and collections
- **Idempotency**:
  - Running the same playbook multiple times leads to the **same resulting state**
  - Essential for safe re-runs

4. Ansible Architecture

- **Control Node**:
  - Where Ansible is installed
  - Runs `ansible` and `ansible-playbook` commands
- **Managed Nodes**:
  - Servers being configured (e.g., EC2 instances)
- Communication:
  - **SSH** (port 22)
- Ansible:
  - Sends small Python modules to targeted nodes
  - Executes modules, then removes them

SSH & Authentication

- Use **public/private key** authentication:
  - Private key on control node
  - Public key in `~/.ssh/authorized_keys` on managed node
- Example SSH command:

```
ssh -i ~/.ssh/tdtu-devops-key ubuntu@54.179.10.25
```

- Ansible inventory:

```
[webservers]
web1 ansible_host=1.2.3.4 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/key
```

How Ansible Executes Tasks

- Execution model:
  i. Read inventory
  ii. Read playbook
  iii. Determine which hosts match `hosts:`
  iv. Connect to hosts via SSH
  v. Execute modules for each task
  vi. Report results:
    - **ok** (no change needed)
    - **changed** (state modified) or **failed**
- Parallelism: Default `forks` controls how many hosts run in parallel

Idempotency in Ansible

- Modules are designed to:
  - Only change state when necessary
- Example:

```yaml
- name: Ensure nginx is installed
  apt:
    name: nginx
    state: present
```

- First run: Installs nginx → `changed`
- Next runs: nginx already installed → `ok`
- This property: Makes it safe to run playbooks regularly (e.g., via CI/CD)

5. Working With Ansible Inventory

- Inventory describes:
  - Hostnames or IPs
  - Groups of hosts
  - Connection details and variables
- Formats:
  - INI format (traditional)
  - YAML format (newer, more structured)
  - Dynamic inventory (scripts/plugins)

Static Inventory (INI Format)

- Example `inventory.ini`:

```
[webservers]
web1 ansible_host=54.179.10.25 ansible_user=ubuntu
web2 ansible_host=54.179.10.26 ansible_user=ubuntu

[dbservers]
db1 ansible_host=10.0.1.10 ansible_user=ubuntu
```

- Notes:
  - `web1`, `web2` are internal names
  - `ansible_host` is actual IP
  - User and SSH key can be configured per host or group

YAML Inventory Example

- `hosts.yaml`:

```yaml
all:
  children:
    webservers:
      hosts:
        web1:
          ansible_host: 54.179.10.25
          ansible_user: ubuntu
    dbservers:
      hosts:
        db1:
          ansible_host: 10.0.1.10
          ansible_user: ubuntu
```

- Command: `ansible -i hosts.yaml webservers -m ping`

Groups & Host Variables

- Group variables (`group_vars/webservers.yml`):

```yaml
ansible_user: ubuntu
ansible_ssh_private_key_file: ~/.ssh/tdtu-devops-key
app_env: staging
```

- Host variables (`host_vars/web1.yml`):

```yaml
app_port: 3000
```

- Benefits: Clean separation between shared and host-specific config

Dynamic Inventory (Overview)

- For cloud environments, IPs change often:
  - Manual inventory becomes painful
- Dynamic inventory:
  - Uses plugins to fetch hosts from AWS, Azure, etc.
- AWS EC2 plugin:
  - Queries EC2, filters by tags, builds inventory automatically
- For this course:
  - Simpler approach:
    - Terraform writes IPs to a file
    - Small script generates `inventory.ini`

Terraform → Ansible Inventory

- Terraform outputs:

```
output "web_ips" {
  value = aws_instance.web[*].public_ip
}
```

- After `terraform apply`:

```
terraform output -json web_ips > web_ips.json
```

- A simple script (Bash or Python) can:
  - Read JSON
  - Generate `inventory.ini` with correct IPs and Ansible variables
- This connects **IaC** and **CM** pipelines

6. Writing Your First Ansible Playbook

- Minimal playbook `site.yml`:

```yaml
- hosts: webservers
  become: yes
  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install nginx
      apt:
        name: nginx
        state: present
```

- Run: `ansible-playbook -i inventory.ini site.yml`

Playbook Structure

- Main keys:
  - `hosts`: target group, `become`: whether to use sudo
  - `vars`: play-level variables, `tasks`: list of tasks
  - `handlers`: defined at play level

```yaml
- hosts: webservers
  become: yes
  vars:
    app_port: 3000
  tasks:
    - name: Install curl
      apt:
        name: curl
        state: present
```

Registering Variables

- Store command/module output:

```yaml
- name: Check Node.js version
  command: node -v
  register: node_version

- name: Print Node.js version
  debug:
    var: node_version.stdout
```

- Use cases:
  - Conditional logic
  - Debugging
  - Making decisions in later tasks

Handlers for Service Restarts

- Handlers run **only when notified**:

```yaml
tasks:
  - name: Deploy nginx config
    template:
      src: templates/nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: restart nginx

handlers:
  - name: restart nginx
    service:
      name: nginx
      state: restarted
```

- Ensures: Nginx restarts only when config changes, Avoids unnecessary restarts

7. Ansible Modules – System Packages

- Package modules:
  - `apt` (Debian/Ubuntu), `yum` (RHEL/CentOS), `dnf`, generic `package`
- Example:

```yaml
- name: Install multiple packages
  apt:
    name:
      - git
      - htop
      - unzip
    state: present
    update_cache: yes
```

- Idempotency: Only installs if missing

Users, Groups & File Permissions

- Manage users:

```yaml
- name: Create deploy user
  user:
    name: deploy
    shell: /bin/bash
    groups: sudo
    state: present
```

- Set passwords, SSH keys, home directories similarly
- Best practice: Use dedicated users for applications

Users, Groups & File Permissions

- Set permissions:

```yaml
- name: Ensure app directory exists
  file:
    path: /var/www/booking-api
    state: directory
    owner: deploy
    group: deploy
    mode: "0755"
```

- Principles:
  - **Least privilege**
  - Avoid running apps as `root`

Services & Systemd

- Using `systemd` module:

```yaml
- name: Ensure nginx is enabled and running
  systemd:
    name: nginx
    state: started
    enabled: yes
```

- Can also stop, restart, reload services
- Useful for managing application services

Services & Systemd

- For Spring Boot:
  - Create `/etc/systemd/system/booking-api.service` from template
  - Then:

```yaml
- name: Reload systemd
  systemd:
    daemon_reload: yes

- name: Start booking-api
  systemd:
    name: booking-api
    state: started
    enabled: yes
```

- Ensures app starts on boot, can be managed via systemd

Files, Templates & Synchronization

- **copy** module:

```yaml
- name: Copy static index.html
  copy:
    src: files/index.html
    dest: /var/www/html/index.html
    mode: "0644"
```

- **template** module:
  - Uses Jinja2 to inject variables
- **synchronize**:
  - Rsync-like for large directories (e.g., static assets)

Networking & Firewalls

- Ubuntu UFW example:

```yaml
- name: Allow SSH
  ufw:
    rule: allow
    port: "22"
    proto: tcp

- name: Allow HTTP
  ufw:
    rule: allow
    port: "80"
    proto: tcp
```

- Combine with: Logs and monitoring to investigate access attempts
- Security: Open only necessary ports

Language Runtimes & Repos

- Install Node.js via apt repo:

```yaml
- name: Add NodeSource APT key
  apt_key:
    url: https://deb.nodesource.com/gpgkey/nodesource.gpg.key
    state: present
- name: Add NodeSource repository
  apt_repository:
    repo: "deb https://deb.nodesource.com/node_18.x focal main"
    state: present
- name: Install Node.js 18
  apt:
    name: nodejs
    state: present
    update_cache: yes
```

- Similar patterns for: Java (OpenJDK), Python, Docker

Git & Docker Modules

- Git checkout:

```yaml
- name: Clone booking-api repo
  git:
    repo: "https://github.com/tdtu-devops/booking-api.git"
    dest: /home/deploy/booking-api
    version: main
```

- Docker container:

```yaml
- name: Run booking-api container
  docker_container:
    name: booking-api
    image: mvmanh/booking-api:1.4.1
    state: started
    restart_policy: always
    ports:
      - "80:3000"
```

Utility Modules: command, shell, lineinfile

- `command`:
  - Runs commands without shell expansions
- `shell`:
  - Runs through shell, allows pipes and redirects
- `lineinfile`:
  - Edit specific lines in config files

```yaml
- name: Ensure max open files limit
  lineinfile:
    path: /etc/security/limits.conf
    regexp: "^\* soft nofile"
    line: "* soft nofile 65535"
```

- Best practice: Prefer higher-level modules before falling back to `command` / `shell`

8. Variables, Facts & Templates

- Variables:
  - Defined in:
    - `vars` block
    - `group_vars/`, `host_vars/`
    - Extra vars (`-e`) on CLI
- Facts:
  - Retrieved via `gather_facts: yes` (default)
- Templates:
  - `.j2` files with Jinja2 syntax

Variable Precedence (High-Level)

- From lowest to highest (simplified):
  - role defaults
  - play vars
  - host vars / group vars
  - extra vars (`-e`) – highest
- Good design:
  - Use lower precedence for defaults
  - Override with group_vars for environment
  - Avoid overusing `-e` in production

Example: Jinja2 Template for Nginx

```
server { # templates/booking-api.conf.j2
    listen 80;
    server_name {{ server_name }};
    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

- Playbook snippet:

```
- name: Deploy nginx config for booking-api
  template:
    src: templates/booking-api.conf.j2
    dest: /etc/nginx/sites-available/booking-api.conf
  notify: reload nginx
```

Conditionals & Loops

- Conditionals:

```
- name: Install htop only on Ubuntu
  apt:
    name: htop
    state: present
  when: ansible_facts['os_family'] == "Debian"
```

- Loops:

```
- name: Create application directories
  file:
    path: "/var/www/{{ item }}"
    state: directory
  loop:
    - booking-api
    - booking-admin
```

Facts for Decision-Making

- Facts example:

```yaml
- name: Show OS family
  debug:
    var: ansible_facts['os_family']
```

- Usage:
  - Choose package manager
  - Select correct config file path
- For heterogeneous environments:
  - Essential to write portable playbooks

9. Ansible Roles – Best Practice

- Why roles?
  - Avoid giant playbooks
  - Encapsulate specific responsibilities
  - Reuse across projects
- Role structure (simplified):

```
roles/
  nginx/
    tasks/main.yml
    templates/
    files/
    vars/main.yml
  nodejs/
    tasks/main.yml
```

Role Example: common

```yaml
- name: Update apt cache    # roles/common/tasks/main.yml
  apt:
    update_cache: yes
- name: Install base packages
  apt:
    name:
      - git
      - curl
      - htop
    state: present
```

- Use in playbook:

```yaml
- hosts: all
  become: yes
  roles:
    - common
```

Galaxy Roles vs Custom Roles

- **Galaxy roles**:
  - Community-maintained roles from Ansible Galaxy
  - Good for quick experiments or standard stacks
- **Custom roles**:
  - Tailored to your organization's standards
- Recommendation for course:
  - Students focus on **custom roles**:
    - `common`, `nginx`, `nodejs`, `springboot`, `docker`

10. Ansible + Terraform Integration

- **Terraform**:
  - Creates EC2 instances, security groups, key pairs
  - Outputs IPs and other values
- **Ansible**:
  - Uses those IPs to configure servers:
    - Install runtimes
    - Deploy applications
    - Configure Nginx, firewalls, services
- Combined workflow:
  - Full automation from empty AWS account to running app

Terraform Outputs → Ansible Inventory

- Terraform `outputs.tf`:

```
output "web_public_ips" {
  value = aws_instance.web[*].public_ip
}
```

- After apply:

```
terraform output -json web_public_ips > web_public_ips.json
```

- Script:
  - Generates `inventory.ini`:

```
[webservers]
web1 ansible_host=54.179.10.25 ansible_user=ubuntu
web2 ansible_host=54.179.10.26 ansible_user=ubuntu
```

11. Practical Configuration Scenario: Node.js Server

- Goal:
  - Provision Node.js app server using Ansible
- Steps:
  i. Install Node.js and PM2
  ii. Create app directory
  iii. Copy `.env` file with environment variables
  iv. Install dependencies with `npm install`
  v. Start app via PM2 or systemd
  vi. Configure log rotation for app logs

Node.js Server Playbook Snippet

```yaml
- hosts: node_servers
  become: yes
  vars:
    app_dir: /var/www/booking-api
  tasks:
    - name: Ensure app directory exists
      file:
        path: "{{ app_dir }}"
        state: directory
        owner: deploy
        group: deploy
        mode: "0755"
    - name: Copy environment file
      copy:
        src: files/booking-api.env
        dest: "{{ app_dir }}/.env"
        owner: deploy
        group: deploy
        mode: "0600"
```

Practical Scenario: Spring Boot Server

- Steps:
  i. Install OpenJDK 17 or 21
  ii. Create `/home/ubuntu/app` directory
  iii. Copy `booking-api.jar`
  iv. Create systemd service
  v. Enable and start service
- Ansible ensures:
  - Correct Java version
  - Service starts on boot
  - Least privilege (no root-run JARs)

Spring Boot systemd Service Template

- `templates/booking-api.service.j2`:

```
[Unit]
Description=Booking API Spring Boot Service
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/app
ExecStart=/usr/bin/java -jar /home/ubuntu/app/booking-api.jar \
--spring.profiles.active={{ app_env }}
Restart=always
Environment=JAVA_TOOL_OPTIONS=-XX:+UseContainerSupport

[Install]
WantedBy=multi-user.target
```

Configuring Nginx Reverse Proxy

- Tasks:
  i. Install nginx
  ii. Deploy site configuration from template
  iii. Enable site and test configuration
  iv. Reload nginx only if config changed

```yaml
- name: Install nginx
  apt:
    name: nginx
    state: present
- name: Deploy booking-api nginx config
  template:
    src: templates/booking-api.conf.j2
    dest: /etc/nginx/sites-available/booking-api.conf
  notify: reload nginx
```

Docker-Based Servers with Ansible

- Common pattern:
  - Install Docker and Docker Compose
  - Copy `docker-compose.yml`
  - Start services

Docker-Based Servers with Ansible

- Example:

```yaml
- name: Ensure Docker is installed
  apt:
    name:
      - docker.io
      - docker-compose
    state: present
- name: Deploy docker-compose file
  copy:
    src: files/docker-compose.yml
    dest: /home/ubuntu/app/docker-compose.yml
- name: Start containers
  command: docker-compose up -d
  args:
    chdir: /home/ubuntu/app
```

12. Running Ansible at Scale

- Parallel execution:
  - `forks` setting controls how many hosts run simultaneously
- Limiting concurrency:

```
ansible-playbook -i inventory.ini site.yml --limit webservers --forks 10
```

- Strategy:
  - **linear** (default): host-by-host per task
  - **free**: each host runs ahead as fast as possible

Idempotent Operations & Retries

- Idempotency crucial at scale:
  - Avoids unintended repeated actions
- Handling failures:
  - `max_fail_percentage` and `any_errors_fatal`
- Retries:

```yaml
- name: Wait for application to become healthy
  uri:
    url: "http://localhost:3000/health"
    status_code: 200
  register: result
  retries: 10
  delay: 5
  until: result.status == 200
```

Advanced SSH Configuration

- Optimize SSH:
  - Use `~/.ssh/config`:

```
Host web-*
  User ubuntu
  IdentityFile ~/.ssh/tdtu-devops-key
```

- In ansible.cfg:

```
[ssh_connection]
pipelining = True
control_path = ~/.ansible/cp/%%h-%%p-%%r
```

- Benefits:
  - Faster runs
  - Less SSH overhead

13. Error Handling, Debugging & Testing

- Debug module:

```yaml
- name: Print variable
  debug:
    var: app_dir
```

- Dry run (check mode):

```
ansible-playbook site.yml --check
```

- Show diffs:

```
ansible-playbook site.yml --diff
```

Common Pitfalls

- Permissions:
  - Forgetting `become: yes` when changing system files
- Environment variables:
  - Services don't see environment from `.bashrc`
  - Use systemd `Environment=` or `.env` files
- PATH issues:
  - Non-login shells may not load `/etc/profile`
- Missing sudo privileges:
  - Ensure Ansible user is in `sudo` group with proper `sudoers` rules

14. Hands-On Labs Overview

- **Lab 1** — Install and configure Ansible, test connectivity
- **Lab 2** — Terraform to create EC2 + outputs for inventory
- **Lab 3** — Basic server configuration (packages, UFW, users)
- **Lab 4** — Provision Node.js or Spring Boot server
- **Lab 5** — Configure Nginx reverse proxy
- **Lab 6** — Deploy Dockerized application using Ansible

Lab 1 — Install and Configure Ansible

- Tasks:
  - Install Ansible on control node
  - Create `inventory.ini` with at least one VM
  - Test connection:

    ```
    ansible -i inventory.ini all -m ping
    ```

- Expected result:
  - `pong` from all reachable hosts

Lab 2 — Terraform + EC2 + Inventory Generation

- Tasks:
  - Use Terraform to:
    - Create 1–2 EC2 instances
    - Output public IPs
  - Write a small script to generate `inventory.ini`
- Outcome:
  - Students see IaC + CM pipeline end-to-end

Lab 3 — Basic Server Configuration

- Tasks:
  - Update apt cache
  - Install common tools (git, curl, htop)
  - Configure UFW to allow SSH and HTTP
  - Create `deploy` user with sudo privileges
- Focus:
  - Idempotency and least privilege

Lab 4 — Provision Node.js or Spring Boot Server

- Tasks:
  - For Node.js:
    - Install Node.js, PM2
    - Deploy app code and `.env`
    - Start PM2 process
  - For Spring Boot:
    - Install OpenJDK
    - Upload JAR
    - Configure systemd service
- Outcome: Running backend API managed by Ansible

Lab 5 — Configure Nginx Reverse Proxy

- Tasks:
  - Install Nginx
  - Deploy app-specific reverse proxy config from template
  - Reload Nginx via handler
  - Access app via public IP (or configured DNS)
- Key learning:
  - Template usage and handler notifications

Lab 6 — Deploy Dockerized Application

- Tasks:
  - Install Docker and Docker Compose
  - Copy `docker-compose.yml`
  - Start containers with `docker-compose up -d`
  - Update image version and re-deploy
- Emphasis:
  - How CM interacts with container-based deployments

15. Best Practices in Configuration Management

- Keep playbooks **idempotent**
- Prefer high-level modules:
  - Avoid `command` and `shell` unless no module exists
- Use **templates** instead of manual file edits
- Organize logic into **roles**
- Separate variables per environment:
  - `group_vars/dev`, `group_vars/staging`, `group_vars/prod`
- Never store secrets in plain YAML:
  - Use Ansible Vault or external secret stores
- Everything in version control: Playbooks, roles, ansible.cfg, inventories

Summary & Next Steps

- Today's focus:
  - Why configuration management matters
  - How Ansible works and its architecture
  - From Terraform-provisioned servers to fully configured app environments
- Next steps:
  - Integrate Ansible runs into CI/CD pipelines
  - Combine with monitoring and security tools
- Key message:
  - **Automate configuration early** to avoid fragile, snowflake servers.